Industrial Defect Detection & Localization

Project Implementation Roadmap

Project Goal

Develop an automated vision-based industrial inspection system that combines classical computer vision, supervised deep learning, unsupervised anomaly detection, visual explainability, and an interactive deployment interface. The project uses the MVTec Anomaly Detection dataset as the primary evaluation dataset.

Overall Workflow

1	**MVTec AD Dataset**	Collect and organize normal, defective, and ground-truth mask images.
2	**Dataset Exploration**	Inspect classes, defect types, image sizes, imbalance, and masks.
3	**Preprocessing**	Resize, noise reduction, CLAHE, ROI extraction, normalization, augmentation.
4	**Classical CV Baseline**	Canny + intensity histograms + GLCM features → Random Forest.
5	**Supervised Deep Learning**	Fine-tune pretrained EfficientNet-B0 for defect classification.
6	**Anomaly Detection**	Train PatchCore using normal images and generate anomaly scores/maps.
7	**Explainability & Localization**	Use Grad-CAM and anomaly heatmaps to visualize defect regions.
8	**Evaluation**	Compare classification, anomaly detection, localization, and efficiency metrics.
9	**Deployment**	Build a Streamlit dashboard with PASS/FAIL results and CSV audit logging.

Phase-by-Phase Roadmap

Phase 1 — Repository & Environment Setup

Create the shared GitHub repository, define the project folder structure, configure .gitignore, create the Python environment, and prepare requirements.txt. Do not commit the MVTec dataset or large trained model files.

Deliverable: reproducible repository that all teammates can clone and run.

Phase 2 — Dataset Exploration

Download and organize MVTec AD. Explore normal and defective images, defect categories, image dimensions, class imbalance, and available ground-truth segmentation masks. Create visual samples for each selected product category.

Deliverable: 01_dataset_exploration.ipynb with dataset statistics and visualizations.

Phase 3 — Preprocessing & Augmentation

Implement resizing, noise reduction, CLAHE, dynamic ROI extraction, and normalization. Build an Albumentations augmentation pipeline using rotations, elastic distortion, Gaussian noise, and brightness/contrast jitter.

Deliverable: reusable preprocessing.py and verified before/after visualizations.

Phase 4 — Classical Computer Vision Baseline

Extract handcrafted features using Canny edge-density distributions, intensity histograms, and GLCM texture statistics. Train a Random Forest classifier to establish an empirical baseline.

Deliverable: baseline metrics and confusion matrix for comparison with deep models.

Phase 5 — EfficientNet-B0 Classification

Fine-tune a pretrained EfficientNet-B0 using transfer learning. Train using AdamW with cosine-annealing learning-rate scheduling. Evaluate precision, recall, F1-score, confusion matrices, and macro ROC-AUC, with particular attention to defect recall.

Deliverable: trained supervised classifier, saved metrics, plots, and model checkpoint.

Phase 6 — PatchCore Anomaly Detection

Train PatchCore exclusively on non-defective images using pretrained feature embeddings and a coreset memory bank. At inference time, calculate anomaly scores and spatial anomaly maps for previously unseen samples.

Deliverable: anomaly detector evaluated using AUROC and qualitative anomaly-map examples.

Phase 7 — Explainability & Localization

Integrate Grad-CAM with the supervised model to produce activation heatmaps and overlay them on inspection images. Compare these explanations with PatchCore anomaly maps and available ground-truth masks.

Deliverable: visual defect-localization results suitable for the final report and demo.

Phase 8 — Evaluation & Benchmarking

Compare the classical baseline, EfficientNet-B0, and PatchCore according to their appropriate metrics. Analyze detection reliability, localization quality, generalization to anomalies, and inference efficiency.

Deliverable: consolidated results tables, plots, error analysis, and conclusions.

Phase 9 — Streamlit Deployment

Develop a Streamlit application supporting single-image and batch inspection. Display PASS/FAIL verdicts, confidence/anomaly scores, visual heatmaps, and persistent CSV audit logs.

Deliverable: end-to-end demonstrable industrial inspection application.

Recommended Team Workflow

All team members should first work from the same dataset loader and preprocessing pipeline. Only after this shared foundation is merged into the main branch should modelling work be split among team members. Use task-based Git branches such as feature/preprocessing, feature/random-forest, feature/efficientnet, feature/patchcore, and feature/streamlit, then merge through pull requests.

Immediate Milestone — Start Here

1	Create/clone the shared GitHub repository
2	Set up the Python environment and requirements.txt
3	Download MVTec AD locally into data/mvtec/
4	Write the dataset exploration notebook
5	Visualize normal, defective, and ground-truth mask samples
6	Implement the common preprocessing pipeline
7	Verify preprocessing and augmentation visually
8	Commit the shared foundation before splitting model work

Suggested implementation order: Dataset → Preprocessing → Random Forest → EfficientNet-B0 → PatchCore → Grad-CAM → Evaluation → Streamlit.